

```
Books/
├── backend/
│   ├── config/
│   │   └── db.js
│   ├── models/
│   │   └── Book.js
│   ├── routes/
│   │   └── bookRoutes.js
│   └── server.js
├── frontend/
│   ├── index.html
│   ├── vite.config.js
│   ├── package.json
│   └── src/
│       ├── App.jsx
│       ├── main.jsx
│       └── components/
│           └── BookList.jsx
```

Navigate to your **Books** project folder and create backend

```
cd C:\Users\Admin\Desktop
mkdir Books
cd Books
mkdir backend
cd backend
npm init -y
```

Install dependencies

```
npm install express mongoose cors dotenv
```

### Create folders and files

```
mkdir models routes  
touch server.js .env
```

## Configure .env

### backend/.env

```
PORT=5000  
MONGO_URI=mongodb://127.0.0.1:27017/book_management
```

## server.js (Main backend file)

```
import express from 'express';  
import mongoose from 'mongoose';  
import cors from 'cors';  
import dotenv from 'dotenv';  
import bookRoutes from './routes/bookRoutes.js';  
  
dotenv.config();  
const app = express();  
  
app.use(cors());  
app.use(express.json());  
app.use('/api/books', bookRoutes);  
  
mongoose.connect(process.env.MONGO_URI, {  
  useNewUrlParser: true,  
  useUnifiedTopology: true,  
})  
.then(() => console.log('✅ MongoDB connected'))  
.catch(err => console.error('❌ Mongo Error:', err));  
  
const PORT = process.env.PORT || 5000;  
app.listen(PORT, () => {  
  console.log(`🚀 Server running at http://localhost:${PORT}`);  
});
```

```
});
```

---

## 4. Mongoose Model

### backend/models/Book.js

```
import mongoose from 'mongoose';

const bookSchema = new mongoose.Schema({
  title: String,
  author: String,
  available: { type: Boolean, default: true },
  bookedBy: { type: String, default: '' }
});

const Book = mongoose.model('Book', bookSchema);
export default Book;
```

---

## 5. Routes

### backend/routes/bookRoutes.js

```
import express from 'express';
import Book from '../models/Book.js';

const router = express.Router();

router.get('/', async (req, res) => {
  const books = await Book.find();
  res.json(books);
});

router.post('/', async (req, res) => {
  const { title, author } = req.body;
  const book = new Book({ title, author });
  await book.save();
});
```

```
    res.status(201).json(book);
  });

  router.put('/:id/book', async (req, res) => {
    const { id } = req.params;
    const { bookedBy } = req.body;
    const book = await Book.findByIdAndUpdate(id, {
      available: false,
      bookedBy
    }, { new: true });
    res.json(book);
  });

  export default router;
```

## Create frontend in Books folder

```
cd ..
npm create vite@latest frontend --template react
cd frontend
npm install
```

## \*\*\*\*\*Run Backend\*\*\*\*\*

```
node server.js
```

If you want auto-reload on changes:

```
npm install --save-dev nodemon
npx nodemon server.js
```

If tailwind gets installed like for installation and set up

## Install Tailwind CSS

```
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

### **tailwind.config.js**

```
/** @type {import('tailwindcss').Config} */
export default {
  content: [
    './index.html',
    './src/**/*.{js,ts,jsx,tsx}',
  ],
  theme: {
    extend: {},
  },
  plugins: [],
}
```

### **src/index.css**

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

In `main.jsx`, import it:

```
import './index.css';
```

## Add Proxy to frontend

In `vite.config.js`:

```
js
CopyEdit
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
```

```
plugins: [react()],
server: {
  proxy: {
    '/api': 'http://localhost:5000'
  }
}
})
```

Now, you can call `/api/books` directly from frontend.

In `index.css`, you can start adding your custom styles. Here's a simple example `/*`

```
index.css */
```

```
body {
  font-family: Arial, sans-serif;
  background-color: #f4f4f4;
  margin: 0;
  padding: 0;
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
}

.container {
  width: 80%;
  max-width: 1200px;
  margin: 20px;
  padding: 20px;
  background-color: white;
  box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
}

h1 {
  color: #333;
}

.button {
```

```
background-color: #007bff;  
color: white;  
padding: 10px 20px;  
border: none;  
border-radius: 5px;  
cursor: pointer;  
}
```

```
.button:hover {  
  background-color: #0056b3;  
}
```

1.

---

## Step 2: Import the CSS into Your React Application

Make sure to import your `index.css` in `src/main.jsx` (or `src/index.jsx` if you're using JSX) to apply the styles globally.

```
// src/index.js or src/index.jsx  
import React from "react";  
import ReactDOM from "react-dom/client";  
import "./styles.css"; // Importing your custom CSS  
import App from "./App";  
  
ReactDOM.createRoot(document.getElementById("root")).render(  
  <React.StrictMode>  
    <App />  
  </React.StrictMode>  
);
```

This will apply the styles defined in `styles.css` to the entire app.

---

## Step 3: Build Your Components

Now, you can proceed with building your components with the custom styles. For example:

```
jsx  
CopyEdit  
// src/components/BookingForm.jsx
```

```
import React from "react";

const BookingForm = () => {
  return (
    <div className="container">
      <h1>Book Your Spot</h1>
      <form>
        <input type="text" placeholder="Name"
className="input-field" />
        <input type="email" placeholder="Email"
className="input-field" />
        <button className="button">Book Now</button>
      </form>
    </div>
  );
};

export default BookingForm;
```

And in your main **App.jsx**:

```
// src/App.jsx
import React from "react";
import BookingForm from "../components/BookingForm";

const App = () => {
  return (
    <div>
      <BookingForm />
    </div>
  );
};

export default App;
```

**\*\*\*\*\*Run Backend\*\*\*\*\***

```
node server.js
```



If you want auto-reload on changes:

```
npm install --save-dev nodemon  
npx nodemon server.js
```

RUN Frontend

```
npm run dev
```